

TRACECHAIN PLATFORM ARCHITECTURE

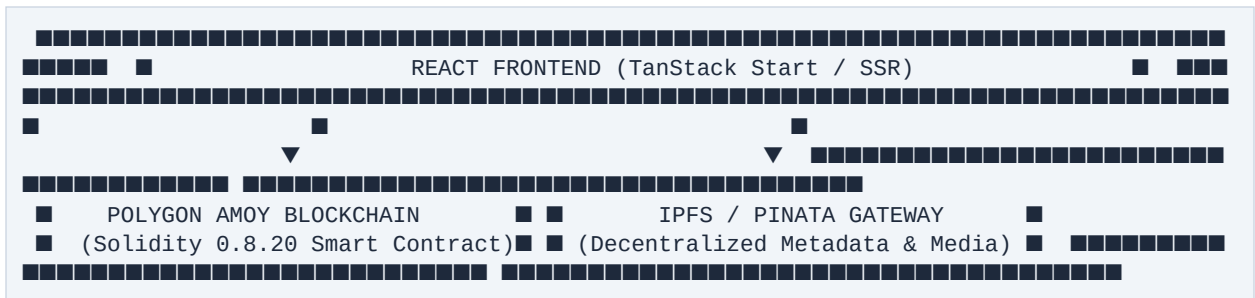
Technical Blueprint, Smart Contract State Machine & Real-World Supply Chain Case Study

1. Executive Summary & Value Proposition

Traditional supply chains suffer from systemic opacity, fraudulent documentation, counterfeit goods, and fragmented custody logs. **TraceChain** resolves these challenges by combining the **Polygon Blockchain (Amoy Testnet)** with **IPFS (Pinata Gateway)** to deliver an immutable, scannable, and verifiable chain of custody for enterprise products.

Key Pillar	Technical & Business Impact
Immutability	Every ownership transfer and status update is signed via MetaMask and permanently recorded on Polygon.
Decentralized IPFS	Product images, lab tests, and specifications are stored on IPFS via Pinata, maintaining low gas costs while ensuring data integrity via CIDs.
Role-Based Control	AccessControl defines strict permissions for MANUFACTURER, DISTRIBUTOR, RETAILER, and ADMIN roles.
State Machine	Smart contract prevents invalid status resets, out-of-order transitions, or modification of delivered items.
Consumer QR Scan	End consumers scan product QR codes to view the complete Polygonscan event history without requiring a crypto wallet.

2. System Architecture & Tech Stack



3. Role-Based Access Control (RBAC) Matrix

Role	Grant Permission	Allowed Actions
DEFAULT_ADMIN_ROLE	Deployer	Grants and revokes supply chain roles.
MANUFACTURER_ROLE	Admin Only	Registers new products (createProduct), pins IPFS metadata.
DISTRIBUTOR_ROLE	Admin Only	Accepts custody, updates status to SHIPPED and IN TRANSIT.
RETAILER_ROLE	Admin Only	Receives final custody, marks status as DELIVERED.
Product Owner	Current Custodian	Only currentOwner can execute status updates or custody transfers.

4. Lifecycle State Machine Rules

Product state transitions follow a strict monotonic progression inside SupplyChain.sol:

CREATED (0) → SHIPPED (1) → IN TRANSIT (2) → DELIVERED (3) [Terminal]

*(Any non-delivered state can also transition directly to **CANCELLED (4) [Terminal]**)*

5. Real-World Case Study: Colombian Specialty Coffee #042

To illustrate the complete operational flow of TraceChain, consider the journey of **Specialty Organic Arabica Coffee Beans (Batch #042)** harvested in Huila, Colombia and delivered to an artisan coffee shop in Manhattan, New York City.

Step 1: Product Registration	Actor: El Paraíso Coffee Estate (Manufacturer) - Harvests 500kg organic coffee beans in Huila, Colombia. - Pins farm photos & lab certificates to IPFS: bafybeic7x2... - Calls createProduct('bafybeic7x2...') on Polygon Amoy. Result: Product #042 created in status CREATED (0).
Step 2: Shipping & Port Transfer	Actor: Andes Global Logistics (Distributor) - El Paraíso transfers ownership to Andes Logistics. - Andes Logistics signs updateStatus(42, SHIPPED, 'Port of Cartagena'). Result: Status updated to SHIPPED (1).
Step 3: Maritime Freight & Customs	Actor: Andes Global Logistics (Distributor) - Cargo container arrives at Port of Miami Customs Facility. - Signs updateStatus(42, IN_TRANSIT, 'Customs Facility, Miami, FL'). Result: Status updated to IN TRANSIT (2).
Step 4: Final Retail Delivery	Actor: Roast & Brew NYC (Retailer) - Beans arrive at Manhattan cafe store. - Ownership transferred to Roast & Brew NYC. - Signs updateStatus(42, DELIVERED, 'Roast & Brew, NYC'). Result: Product status locks in DELIVERED (3).
Step 5: Consumer QR Code Scan	Actor: End Consumer - Scans QR code on coffee bag in Manhattan cafe. - Opens /products/42 and inspects full provenance certificate, harvest CID, and verified Polygonscan transaction log.

6. Dockerization & Production Setup

TraceChain is packaged with multi-stage Docker builds for zero-dependency container deployment. The stack includes production environment variables, gas fee optimizations, and SSR server functions.

Container File	Description & Target
Dockerfile	Multi-stage build compiling TypeScript/TanStack Start assets and serving via Node runtime on port 8080.
docker-compose.yml	Orchestrates container deployment with environment variable bindings for Pinata IPFS keys.
.dockerignore	Excludes node_modules, build outputs, and local secrets from container context.